WEEK 05 · RAZOR PAGES FORM FLOW

讓表單
真的收到資料

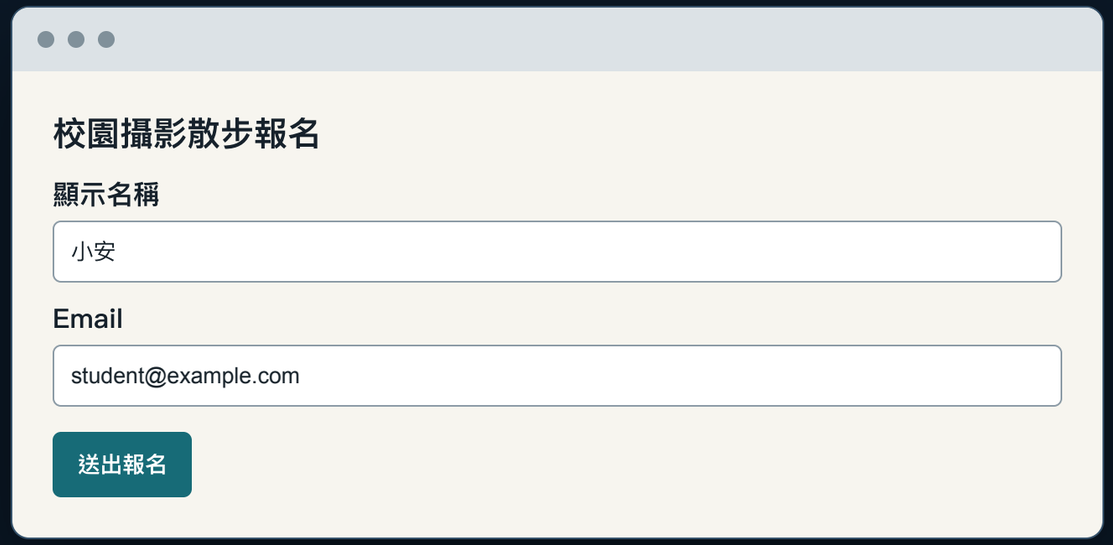
小安的活動網站已經好看了，但同學按下「送出」後，資料要去哪裡？今天把畫面接上 ASP.NET 後端，完成第一條看得見的後端資料路線。

下載學生任務包 ↓　下載 PDF ↓



EXPECTED RESULT · 先看本週終點

今天先證明一件事：畫面輸入真的進入 C#。



ASP.NET 回應

報名資料已收到

參加者：小安

本週界線：先做到「收到與回應」；資料規則留到第六週。

開啟第五週成功路線 Demo ↗

MISSION MAP

三節課，只完成「收到資料」這一條路。

1

建立與認路

建立 Razor Pages、跑出 localhost，只認畫面、C#、CSS。

2

做出 POST 表單

理解 form、label、asp-for，完成可操作的報名表。

3

讓後端收到

用 Input、BindProperty 與 OnPost，顯示成功摘要。

> 第五週不教驗證規則；第六週再處理「收到不等於接受」。

THE WHOLE JOURNEY

本週追蹤一筆資料經過五個站。



下一站　從頭看

使用者先在瀏覽器填入課堂假資料。

這五站會貫穿第五週

ZERO TO ONE · PROJECT START

先不要急著貼表單。
第一個完成點是：自己的 Razor Pages 專案能跑。

1 建立
Razor Pages 專案

→

2 執行
開啟 localhost

→

3 認路
畫面、C#、CSS

→

4 完成
表單、POST、收到

主要路線

使用 Visual Studio 2026 建立 **ASP.NET Core Web App (Razor Pages)**。

卡住時的備援

用 `dotnet new webapp` 建立同一種專案，不換框架、不重做整頁。

開啟從零開始指南 ↗

START 01 · VISUAL STUDIO

建立一個乾淨的 Razor Pages 專案。

1

選範本

「建立新的專案」→ 搜尋 Razor → 選 ASP.NET Core Web App (Razor Pages)。

不要選 MVC、Web API、Blazor 或 Empty。

2

填設定

專案名稱：`Week05Registration`
Framework：.NET 10.0 (LTS)
Authentication：None

保留 HTTPS；其餘先用預設值。

建立位置要記得：請放在自己找得到、可寫入的資料夾；不要直接放進 ZIP 或雲端唯讀預覽。

範本名稱與框架選對，後面才會看到 Pages 資料夾

START 02 · TERMINAL FALLBACK

Visual Studio 找不到範本時，終端機也能建立同一種專案。

```
dotnet --version
dotnet new webapp -n Week05Registration -f net10.0
cd Week05Registration
dotnet watch
```

每一行在做什麼

`--version`：確認 .NET SDK 可用。

`new webapp`：建立 Razor Pages 專案。

`cd`：進入專案資料夾。

`watch`：啟動並在修改後自動重建。

這是備援路線：Visual Studio 已成功建立者不必重做。

若 --version 失敗，先回第二週補安裝，不要繼續貼程式

暫停講解 · 學生實作 0

照五步建立、執行，再確認自己找得到檔案。

25 分鐘

照做步驟

1. 建立 `ASP.NET Core Web App (Razor Pages)`，名稱填 `Week05Registration`。
2. Framework 選 `.NET 10.0`、Authentication 選 None，再建立專案。
3. 先在方案總管展開 `Pages`，找到 `Index.cshtml` 與 `Index.cshtml.cs`。
4. 按 `Ctrl + F5`；或在專案資料夾執行 `dotnet watch`。
5. 打開顯示的 localhost 網址；停止後再啟動一次。

逐項自驗

- 瀏覽器首頁正常，網址以 localhost 開頭。
- 終端機 / Output 沒有 Build error。
- 能指出 `Pages` 是畫面與 C#，`wwwroot` 是靜態檔案。
- 第二次啟動仍能回到同一首頁。

卡住先查：沒看到 Pages，先確認範本不是 MVC、Web API、Blazor 或 Empty。

第 6 分鐘：還沒看到 localhost 的先舉手，不先貼表單。

快學生：指出 Index 兩檔與 site.css，但不先改內容。

逐步指南 ↗

時間到先看 localhost；未成功者結伴排查建立與啟動步驟

PROJECT MAP · 先認路，不用背

今天只要知道：畫面、C#、CSS 在哪裡。

```
Week05Registration/
├─ Pages/
│  ├─ Index.cshtml
│  └─ Index.cshtml.cs
└─ wwwroot/
   └─ css/site.css
```

今天只在三處移動

改畫面：`Pages/*.cshtml`

處理資料：`Pages/*.cshtml.cs`

改樣式：`wwwroot/css/`

先認路：`Index.cshtml` 與 `Index.cshtml.cs` 是同一個 Razor Page 的合作夥伴。其他檔案先知道存在，未來用到再介紹。

POINT 01 · GET VS POST

GET 是拿報名頁，
POST 是交出欄位資料。

第一次開啟首頁，瀏覽器向伺服器拿 HTML；填完按下送出，才把 form 內的資料交給後端。

GET / 拿頁面

```
瀏覽器 ── GET / ──→ 伺服器
瀏覽器 ←── 表單 HTML ──
```

對應 Razor Pages 的 **OnGet**。

POST / 交資料

```
瀏覽器 ── POST / ─→ 伺服器
          名稱、Email、場次
```

對應今天要完成的 **OnPost**。

POINT 02 · FORM BOUNDARY

form 像一個資料信封，只有信封裡的欄位會一起送出。

```
<form method="post">
  <label>顯示名稱</label>
  <input ...>

  <button type="submit">
    送出報名
  </button>
</form>
```

先預測

如果 input 放在 </form> 後面，按下送出時，它會一起送嗎？

```
</form>
<input name="note"> <!-- Form 表單外 -->
```

不會

會

先選答案，再用「信封範圍」解釋。

method="post" 描述這次送出的方式

POINT 03 · LABEL BEFORE STYLE

label 不是裝飾，
它把「問題」和「輸入位置」連起來。




```
<label for="nameDemo">
  顯示名稱
</label>
<input id="nameDemo">
```

Placeholder 只是範例，輸入後會消失；不能取代 label。

POINT 04 · FIELD NAME

後端屬性還沒建立前，
先用 name 標出資料的送件地址。

```
Pages/
├─ Index.cshtml        ← 先完成可編譯表單
└─ Index.cshtml.cs     ← 下一輪建立 Input
```

```
<label for="displayName">顯示名稱</label>
<input id="displayName"
       name="Input.DisplayName">
```

label

for → id

+

送出名稱

name → 後端地址

第三節建立 Input 後，再把這兩行改成 `asp-for`。

每個停點都必須能編譯；不先引用不存在的屬性

暫停講解 · 學生實作

PAUSE & PRACTICE 01

只改 Index.cshtml，每完成一段就 Build 一次。

25 分鐘

查看本週成功路線 Demo ↗

照做步驟

1. 開 `Pages/Index.cshtml`，保留頂端 `@page`、`@model`。
2. 打開 `Index-form-week05-starter.txt`，只複製表單區到標記位置。
3. 確認 `<form method="post">` 包住四個欄位與 submit 按鈕。
4. 逐欄檢查 `label for=input id`。
5. 逐欄填 `name="Input.屬性"`，儲存、Build，再開 localhost 操作。

操作與畫面自驗

- Build 為 0 errors，首頁顯示名稱、Email、場次、備註。
- 按 Tab 依序走過欄位與送出按鈕。
- 點每個 label，游標會進入正確欄位。
- 縮到 390px 沒有整頁水平捲動。

卡住先查：若出現兩個頁面框架，刪掉複製來的 html/head/body，不刪 @page。

第 12 分鐘：先保留名稱、Email、場次三欄，確認 form 與 label。

快學生：改善 label 與輸入框間距，不新增功能。

☐ 我已 Build、鍵盤走訪、點 label，並在自己的 localhost 看過表單。

時間到：抽 2 位同學指出 form 範圍、for / id 與 name

CHECKPOINT 01 · 學生操作

照畫面順序檢查，不要只看「好像有表單」。

1

看檔案頂端

確認仍有 @page、@model，而且沒有第二組 html/head/body；@model 此刻只保留，下一頁再拆解。

2

框出 form

從 <form method="post"> 畫到 </form>，四個欄位與按鈕都要在框內。

3

逐欄讀地址

點 label 驗證 for / id；再讀出每個 name="Input.屬性"。

最後按 Ctrl+F5：表單可操作且沒有 Build error 才過關；此時送出只重新載入是合理的。
哪項失敗就修哪裡：頂端缺字就還原 @page / @model；欄位在 form 外就移回框內；點 label 沒反應就修正 for / id；Build 失敗只先修第一個錯誤。

POINT 05 · PAGE MODEL

小寫 @model 宣告類別；
大寫 Model 是這頁拿到的物件。

```
// Index.cshtml
@model IndexModel

@Model.Input.DisplayName

// Index.cshtml.cs
public class IndexModel : PageModel
```

1. `@model`：宣告這頁搭配哪個後端類別。
2. `IndexModel`：在 `Index.cshtml.cs` 定義的類別。
3. `Model`：Razor 執行時實際拿到的 `IndexModel` 物件。

先記大小寫：`@model` 是指令，`Model` 是物件。

兩個檔案透過 @model IndexModel 合作完成一頁

POINT 06 · INPUT MODEL · STEP BY STEP

RegistrationInput 是資料格式，還不是這次送來的資料。

```
public class RegistrationInput
{
    public string DisplayName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Session { get; set; } = string.Empty;
    public string? Note { get; set; }
}
```

1. 類別＝空白格式

它規定報名資料有哪四個欄位。

2. 放在正確位置

到 Pages/Index.cshtml.cs，放在 IndexModel 的**上方**，不要塞進 OnGet。

3. 儲存並確認

建置應為 0 errors；這頁只建立類別，下一頁才建立 Input 物件。

類別描述資料長相；物件才會裝進這一次的值

POINT 07 · MODEL BINDING · STEP BY STEP

在 IndexModel 裡建立 Input，準備接住表單 POST 的資料。

```
// Pages/Index.cshtml.cs 頂端
using Microsoft.AspNetCore.Mvc;

public class IndexModel : PageModel
{
    // ① 允許 POST 欄位綁定到 Input
    [BindProperty]
    // ② public：框架與頁面程式可存取
    // ③ RegistrationInput：型別 / 資料格式
    // ④ Input：IndexModel 裡的屬性名稱
    // ⑤ get; set;：允許讀取與寫入
    // ⑥ new()：先建立物件，避免 Input 尚未指向物件
    public RegistrationInput Input
        { get; set; } = new();
}
```

1. 開啟檔案

Pages/Index.cshtml.cs；先確認上一頁的 RegistrationInput 已在 IndexModel 上方。

2. 找到插入位置

找到 public class IndexModel : PageModel，把 Input 屬性加在類別大括號內；不要放進 OnGet。

3. 處理紅線

若 BindProperty 有紅線，在檔案頂端加入 using Microsoft.AspNetCore.Mvc;。

4. Build 自驗

Build 應為 0 errors。畫面暫時沒變是正常的；下一頁才加入 OnPost()。

「小安」實際怎麼走：asp-for="Input.DisplayName" → HTML name="Input.DisplayName" → POST Input.DisplayName=小安 → Model Binding → Input.DisplayName。

完成標準：位置正確、0 errors，並能指出下一頁才處理 OnPost

POINT 08 · HANDLER · STEP BY STEP

在 IndexModel 裡加入 OnPost()，接住使用者送出的表單。

```
public class IndexModel : PageModel
{
    [BindProperty]
    public RegistrationInput Input { get; set; } = new();

    public bool Submitted { get; set; }

    public void OnPost()
    {
        Submitted = true;
    }
}
```

1. 開啟並定位

開 Pages/Index.cshtml.cs；在 IndexModel 裡、Input 屬性下方加入程式，不要放進 RegistrationInput。

2. 加入完成狀態

Submitted 是布林值；一開始是 false，送出後改成 true。

3. 分清觸發時機

第一次開頁面呼叫 OnGet()；<form method="post"> 送出時，依命名慣例自動呼叫 OnPost()。

4. Build 自驗

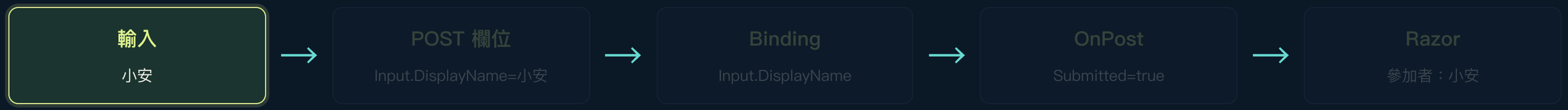
Build 應為 0 errors。若還看不到摘要是正常的；第 21 頁才加入讀取 Submitted 的 Razor 區塊。

目前流程：form POST → OnPost() → Submitted = true → Razor 決定顯示摘要。此時尚未驗證內容，空白資料也可能成功。

完成標準：位置正確、0 errors，並能說出 OnGet 與 OnPost 的觸發時機

LIVE TRACE

「小安」如何走到成功摘要？



顯示下一步　重新追蹤

第一站：使用者在瀏覽器輸入「小安」。

每一站都可觀察，除錯才有方向

POINT 09 · RESPONSE · STEP BY STEP

在 Index.cshtml 加入結果區塊，讓送出後的「小安」真的出現在畫面上。

```
@* Pages/Index.cshtml：放在 </form> 下方 *@
@if (Model.Submitted)
{
    <section class="alert alert-success">
        <strong>報名資料已收到</strong>
        <p>參加者：@Model.Input.DisplayName</p>
    </section>
}
```

1. 開啟並定位

開 Pages/Index.cshtml；找到表單結尾 </form>，把結果區塊貼在它的下方，不是貼進 Index.cshtml.cs。

2. 判斷何時顯示

@if (Model.Submitted) 表示：只有 OnPost() 已把 Submitted 改成 true 才顯示摘要。

3. 讀出姓名

Model 是這頁的 IndexModel；Input 是表單資料屬性；DisplayName 是姓名欄位。

4. 執行自驗

啟動網站，輸入「小安」並送出；送出前沒有摘要，送出後應看到「參加者：小安」。

畫面為什麼出現：form POST → OnPost() 設為 true → Razor 重新產生頁面 → @if 成立 → 顯示 Input.DisplayName。這仍不代表資料已存入資料庫。

完成標準：送出前不顯示；輸入「小安」送出後顯示參加者：小安

暫停講解 · 學生實作

PAUSE & PRACTICE 02

照檔案順序，把表單正式接上 PageModel。

25 分鐘

查看本週成功路線 Demo ↗

照順序完成

1. `Index.cshtml.cs`：在 IndexModel 上方加入 RegistrationInput。
2. 同檔案：在 IndexModel 內加入 BindProperty 與 OnPost。
3. `Index.cshtml`：四個欄位改接 `asp-for="Input.*"`，並在 form 後加入結果摘要。
4. 在 `OnPost` 的 `Submitted = true;` 設中斷點，用 **F5 / Start Debugging** 啟動；Ctrl+F5 不會停。
5. 送出第一組假資料；停住後查看 `Input.DisplayName`，按 Continue，再送第二組並比對摘要。

每一站都確認

1. 儲存後建置為 0 errors。
2. F5 送出時，中斷點讀得到第一組 `Input.DisplayName`。
3. Continue 後，摘要顯示「參加者：第一組名稱」。
4. 第二組送出後，摘要內容跟著改變。
5. 目前刻意沒有驗證與資料庫。

卡在建置：先看第一個錯誤與對應行，不重貼整頁。

卡在收值：比對 `asp-for` 與 Input 名稱；若環境不能除錯，就以兩組不同摘要作替代證據。

卡在中斷點：確認用 F5 而非 Ctrl+F5，且紅點設在 `OnPost` 內。

- [ ] 我已依檔案順序，用兩組假資料完成 POST 與摘要驗證。

時間到：請沿「檔案 → 動作 → 預期結果」說明

CHECKPOINT 02 · 排錯操作

送出沒有結果時，沿五站逐一打開檢查。

1 form
開 cshtml：method="post"？

→

2 asp-for
路徑與 Input 屬性同名？

→

3 BindProperty
Input 屬性可接收？

→

4 OnPost
設中斷點後有停住？

→

5 Razor
Submitted 成立並顯示摘要？

> 先用一組固定假資料重現，記下「輸入、操作、預期、實際」；每次只修第一個斷點，重新 Build 後再送同一組資料。

現象越具體，修正範圍越小

MISSION COMPLETE · WEEK 05

你完成的不是一張靜態表單，而是第一條後端接收路線。

專案能重啟

localhost 可以停止後再次執行。

兩組資料都會變

輸入不同名稱，成功摘要跟著改變。

能說明五站

瀏覽器 → POST → Binding → OnPost → Razor。

下一週問題：空白名稱與 Email=abc 也可能被收到；收到不代表應該接受。

☐ 我能重新啟動專案 ☐ 表單可操作 ☐ 後端收到兩組不同資料